AISYS | RFID-Enabled Library Management System

FINAL TECHNICAL REPORT | Candidate technical evaluation | 09 November 2026

Evidence-based prototype, implementation boundaries and production design

Repository baseline: cca71ff7 (Phase 6); Phase 7 regression verified 09 November 2026

Executive summary

This report documents a limited working RFID-enabled library management prototype and its proposed production architecture. Implemented capabilities include a React/TypeScript staff interface, FastAPI backend, SQLAlchemy/SQLite persistence, Alembic migrations, authentication and RBAC, catalog and member APIs, circulation, CSV migration staging, mock RFID tagging and scanning, simulated RFID security-state updates, mock inventory reconciliation, renewal/fine policies and a transactional mock integration outbox. All demonstration data is synthetic.

Phase 7 regression verified 114 passing backend Pytest tests, one nonblocking Starlette/httpx deprecation warning, a successful TypeScript/Vite production build, SQLite integrity_check = ok, zero foreign-key violations, and Alembic revision 5c4d888ae5e1.

The system is not a production-certified ILMS or RFID integration. No real RFID devices, NCIP/SIP2 connector, existing ILMS, offline sync, deployed PostgreSQL or production load tests are claimed.

1 | Problem, objectives and scope

Objective: demonstrate a controlled catalogue → physical copy → member → circulation journey and a device-neutral RFID simulation, while documenting requirements and safe interoperability boundaries. The full source baseline contains 12 functional requirements, 9 nonfunctional requirements and 10 acceptance criteria; the prototype delivers only a subset.

- Use synthetic data; never connect to or write into an institutional ILMS without authorization.
- Enforce access controls on backend endpoints, not solely in the browser.
- Separate verified features from proposals, mock boundaries and unimplemented scope.
- Deliver working setup instructions, reproducible evidence and a production gap register.

2 | Requirement traceability and acceptance

The accompanying AISYS_Phase4_Canonical_RTM.xlsx supersedes the Phase 1 progress tracking view for Phase 4 status; the original Phase 1 workbook remains unchanged as historical baseline. Status labels apply to complete source requirements, not isolated subfeatures. “Implemented” requires the full requirement to be supported; several functioning subfeatures remain “Partially implemented”.

Classification	Meaning
Implemented	Entire specified requirement demonstrably delivered and verified
Partially implemented	At least one component running; other stated parts missing or unproven
Design-only	Design or proposed contract exists, without implementation evidence
Unimplemented	No implementation and no corresponding developed design evidenced

The Phase 4 coverage count of 0 fully implemented, 13 partially implemented and 8 design-only requirements is historical and must not be treated as current. Phase 5 and Phase 6 added migration, circulation, integration and inventory functionality. The Phase 7 canonical RTM will provide the reconciled requirement-level coverage. No combined AISYS acceptance scenario is claimed fully accepted without complete scenario-specific evidence.

3 | Architecture and technical decisions

Implemented developer environment: React/TypeScript, FastAPI REST APIs, SQLAlchemy, SQLite and Alembic. Phase 5 added member CSV staging, renewal policies, fine-limit enforcement, mock NCIP/SIP2 adapters and a transactional integration outbox. Phase 6 added RFID security-state simulation and mock inventory reconciliation.

Proposed production architecture: modular monolith, PostgreSQL, isolated LAN/TLS proxy, real reader/gate/smart-card adapters, standards-compliant NCIP/SIP2 connectivity, automatic integration workers and controlled offline deployment. The prototype includes a mock transactional outbox but not automatic delivery or production interoperability.

Layer	Prototype verified	Proposed / not implemented
UI	React basic staff workflows	Full OPAC, advanced search, complete RFID and reporting UI
API	Authentication, catalog, members, circulation, CSV import, RFID and mock inventory	Gate-event, complete reporting and other proposed interfaces
Persistence	SQLite, SQLAlchemy, Alembic, CSV staging, integration outbox	Production PostgreSQL, persistent inventory sessions, institutional restore proof
Device boundary	Synthetic EPC assignment, scan and security-state simulation	Physical readers, gate devices, smart-card hardware and acknowledgements
Operations	Local runbook, health checks, manual mock outbox dispatcher	Offline installer, automatic workers, production monitoring and restore drill

Representative implemented flow

An administrator signs in, creates a bibliographic record, physical item and synthetic member, then checks out and returns an item. The associated ACTIVE synthetic RFID tag is disarmed on checkout and rearmed on return within the circulation transaction. Separate mock endpoints demonstrate EPC assignment, scanning and inventory reconciliation. No physical security-bit programming occurs.

4 | Data model and integrity

The logical design covers bibliographic records, physical copies, members, loans, user/role/permission, sessions, RFID tags, audit events and proposed inventory, gate and migration records. Live prototype schema details must be treated as defined by Alembic and SQLAlchemy models; the Phase 2 data model contains further future tables and constraints that are not necessarily present in SQLite.

- Bibliographic record describes a title; a physical item has accession and circulation state.
- Loan links item and member; returns restore item availability in the demonstrated flow.
- RFID EPC is synthetic and unique within the tested mock assignment flow.
- The prototype includes guarded renewal updates, migration staging, member fine balances and a transactional integration outbox. Proposed production extensions still include comprehensive tag history, persistent stocktake snapshots, hardware acknowledgements, automated workers and independently verified backup/restore.

5 | REST interface and permission model

The implemented backend includes authentication, catalog/item/member APIs, checkout, return, renewal, mock RFID assignment and scan, member CSV import, mock inventory reconciliation, and supporting mock ILMS

integration/outbox services. Refer to current FastAPI OpenAPI output for exact route contracts. Proposed Phase 2 interfaces must not be confused with implemented endpoints.

API demonstration	Evidence boundary
Login/logout and permissions	Tests document authentication and RBAC baseline
Catalog, items and members	List/create APIs with automated tests
Checkout, return and renewal	Backend regression coverage; selected restrictions, auditing and mock outbox
POST /api/v1/rfid/tags/assign	Tested software-only association
POST /api/v1/rfid/scans/mock	Tested lookup and unknown-tag behavior
CSV import and mock inventory	Implemented backend services and automated tests; spreadsheet-scale import and persistent inventory incomplete

6 | Security, privacy and compliance considerations

Prototype security mechanisms: Argon2 password hashes, revocable bearer sessions, permission-based endpoint checks, synthetic data and selected audit entries. Local-process rate limiting is not distributed; database-enforced audit immutability is absent. TLS, secrets management, monitoring and penetration testing need production validation. Prevent .env, local databases, secrets and credentials from entering Git or the submission package.

7 | RFID and integration boundary

RFID functionality is software-only: synthetic EPC association and lookup, transactional security_armed updates for associated ACTIVE tags during checkout and return, and inventory reconciliation against currently AVAILABLE items. Physical readers, device acknowledgements, gate processing, shelf-level misplaced detection and persistent inventory sessions remain unimplemented. Mock NCIP/SIP2 adapters do not implement certified protocols.

8 | Circulation policies and gaps

Checkout, return and renewal are implemented with selected eligibility and fine-limit rules, including reference restrictions. Fine balances are stored in integer INR paise, with a configurable prototype threshold. Mock NCIP/SIP2 outcomes and a durable local outbox are implemented, but no live ILMS transaction or standards certification is claimed. Full patron-card personalization and complete institutional policy acceptance remain outstanding.

9 | Migration and data protection

Phase 5 implemented UTF-8 member CSV validation, duplicate detection, staging, reconciliation, transactional rollback and an authenticated JSON-based import API with CSV content. Direct XLSX import and verified processing of approximately 20,000 supplied spreadsheet records remain outstanding. No external institutional records were migrated; backup/restore acceptance is not complete.

10 | Verification method and results

Check	Recorded outcome	Source
Backend Pytest	114 passed, 0 failed, 1 warning	evidence/phase7/final-verification.md
React build	TypeScript and Vite production build passed	evidence/phase7/final-verification.md
Database	Integrity ok; 0 foreign-key violations; Alembic head 5c4d888ae5e1	evidence/phase7/final-verification.md
Browser API	Login, create records, checkout, return	evidence/phase3/implementation-verification.md
Seed repeat	Second run created no additional records	evidence/phase3/implementation-verification.md

Current evidence includes user-executed Phase 7 backend regression, frontend production compilation and SQLite integrity/Alembic verification. See evidence/phase7/final-verification.md. These checks do not establish production deployment, hardware compatibility, complete UI coverage, performance qualification, or institutional acceptance.

11 | Risk register and production-readiness gaps

Risk / open work	Severity	Recommended follow-up
Legacy records modification	Critical	Read-only contract, sandbox migration, backup/checksum restore
Misstated RFID certification	High	Vendor SDK, real hardware and protocol tests
Concurrency / event delivery	High	Retain existing guarded updates and mock transactional outbox; add production idempotency, concurrency and worker verification
Offline deployment failure	High	Disconnected install-update-rollback drill
Migration loss or duplication	High	Extend implemented CSV staging and reconciliation to supplied spreadsheet-scale testing and verified backup/restore
Security hardening	High	TLS, secrets, distributed throttling, audit immutability
Reporting / inventory completion	Medium	Implement dashboards, persistent stocktakes, shelf-level misplaced detection and handheld confirmation events

12 | Deployment and reproduction

Local Windows reproduction requires Python 3.13, Node/npm and Git. From backend: create .venv using py -3.13 -m venv .venv; install requirements; copy .env.example to .env; run alembic upgrade head; run scripts.seed_auth and scripts.seed_library; start uvicorn app.main:app --reload. From frontend: npm ci; copy .env.example .env; npm run build; npm run dev. See docs/implementation/phase3-prototype-runbook.md for commands and login/demo sequence.

Verify with backend python -m pytest -v, alembic check and pip check; frontend npm run build. FastAPI Swagger is at 127.0.0.1:8000/docs and frontend dev server at localhost:5173. Local SQLite and credentials should remain excluded from Git and archived submission files.

13 | Staged roadmap

Stage	Required follow-up
Immediate demo readiness	Validate branch and current test outputs; capture UI screenshots; correct stale README status
Next engineering iteration	Complete remaining circulation/card policies, device acknowledgements and RFID frontend screens
Integration iteration	Mock gate/CCTV/notification flows; validate NCIP/SIP2 interoperability beyond existing mock adapters
Operational iteration	Spreadsheet-scale migration validation, persistent inventory, reporting, offline package and restore drill
Production qualification	PostgreSQL, concurrency/load/security/UAT, vendor certification, controlled pilot

14 | Submission inventory and limitations

The planned Phase 7 submission package consists of a reconciled technical report (DOCX/PDF), 16-slide presentation, canonical RTM, repository, reproducible demo guide and documented verification evidence. Packaging and final Git synchronization remain subject to completion and verification.

The original seven-page AISYS Candidate Software Development SOP was supplied and reviewed during Phase 7. The report and canonical RTM are aligned to its FR01-FR12, NFR01-NFR09, AC01-AC10 and mandatory deliverable categories. Evaluator acceptance and final submission verification remain outstanding.

Phase 7 | Implemented additions and evidence

Phase 5 introduced member CSV migration staging, duplicate checks, reconciliation, renewal rules, integer-paise fine balances, mock NCIP/SIP2 adapters, and a persistent transactional integration outbox. The dispatcher is manually invoked, supports simulated delivery outcomes and limits retries to three attempts.

Phase 7 | RFID and inventory evidence

Phase 6 introduced transactional security-state changes for associated ACTIVE synthetic RFID tags. The mock inventory API POST /api/v1/inventory/reconcile/mock compares synthetic EPC scans with currently registered ACTIVE tags and AVAILABLE items, identifying matched, missing, unexpected-known and unknown EPCs. Duplicate reads are deduplicated. Persistent inventory sessions and shelf-level misplaced detection are absent.

Phase 7 | Verified regression baseline

On 09 November 2026, the user executed the full backend test suite: 114 passed, zero failed, one nonblocking deprecation warning, in 25.39 seconds. The React/TypeScript Vite production build passed. SQLite integrity_check returned ok, foreign_key_check reported zero violations, and the database revision matched Alembic head 5c4d888ae5e1. These are local developer-environment observations, not institutional UAT.

Phase 7 | Acceptance and compliance boundaries

AC01 remains partial because the implemented CSV migration has not demonstrated the complete supplied spreadsheet-scale workflow. AC03/AC04 have stronger partial evidence from renewal, mock ILMS and fine-policy tests. AC05 remains partial because the mock inventory preview lacks a complete handheld shelf workflow, misplaced-item identification and confirmation events. AC06 gate alarm/CCTV/email workflow and AC09 offline installation/update/rollback have not been demonstrated. AC07 lacks mock smart-card authentication. Full acceptance of any combined AC requires scenario-specific evidence.

Phase 7 | Production completion roadmap

Priority remaining work includes complete dashboard/reporting workflows, mock smart-card and security-gate event flows, notification and camera adapters, spreadsheet-scale migration evidence, documented backup/restore drills, an offline installation/update package, complete operational guides, full UI workflow coverage, hardware/vendor integration, performance/security testing and institutional UAT. PostgreSQL production deployment remains proposed only.

Appendix | Source document register

- docs/requirements/AISYS_Phase1_Requirements_Initiation.docx
- docs/requirements/AISYS_Phase1_RTM_Progress.xlsx
- docs/architecture/system-architecture.md
- docs/database/data-model.md
- docs/api/rest-api-contracts.md
- docs/integrations/integration-design.md
- docs/security/security-and-business-rules.md
- docs/implementation/phase3-prototype-runbook.md
- docs/implementation/phase3-traceability-summary.md
- evidence/phase3/implementation-verification.md
- evidence/phase3/pytest-results.txt

- evidence/phase3/frontend-build.txt